

Rapport Technique : Apprentissage Auto-Supervisé avec SimCLR sur CIFAR-10

1. Introduction

Contexte

L'affirmation croissante de l'apprentissage auto-supervisé repose sur sa capacité à exploiter les données sans étiquettes. Pour des tâches comme la vision par ordinateur, des architectures comme SimCLR ont émergé pour réduire la dépendance aux données labellisées.

Problématique

Comment optimiser les représentations visuelles de données CIFAR-10 sans utiliser d'étiquettes tout en garantissant des performances supervisées robustes ?

Objectifs

1. Mettre en œuvre un modèle basé sur SimCLR intégrant un backbone ResNet-18 spécifique à CIFAR-10.
 2. Évaluer les représentations apprises grâce à une classification supervisée en mode linéaire.
-

2. État de l'art

L'auto-supervision s'appuie sur des contrastes entre des représentations issues de multiples augmentations. SimCLR (Simple Contrastive Learning Representation) repose sur deux idées principales : - Plusieurs augmentations des échantillons. - Optimiser une perte NT-Xent pour rapprocher les paires positives (mêmes images augmentées).

3. Méthodologie

Pipeline SimCLR

1. **Extraction de caractéristiques** : Backbone ResNet-18 ajusté pour les images de taille 32x32 (CIFAR-10).
2. **Projection** : Utilisation d'un MLP (2 couches) pour obtenir des embeddings de dimension 128.
3. **Perte** : NT-Xent Loss utilisée pour maximiser la similitude entre deux vues augmentées tout en minimisant la similitude avec négatifs.

Dataset et Transformations

- **Dataset** : CIFAR-10.
- **Transformations** : Découvrez une vue double augmentée par des techniques telles que recadrage aléatoire, jitters de couleur et flou gaussien.

Entraînement

- Optimiseur : AdamW.
 - Échauffement des courbes de learning-rate grâce à des plages cosines.
-

4. Implémentation

Le SimCLR a été mis en œuvre en se basant sur PyTorch. Voici les fichiers clés :

- **models.py** : Définit l'architecture du modèle avec un backbone ResNet adapté et un projector MLP inclus.
- **train_simclr.py** : Gère l'entraînement contrastif en utilisant la NT-Xent Loss.
- **eval_linear.py** : Évalue les représentations par l'ajout d'un classifieur linéaire, entraîné sur les étiquettes supervisées CIFAR-10.

Structure

```
class SimCLR(nn.Module):  
    def __init__(self, backbone="resnet18", proj_dim=128):  
        super().__init__()  
        ...  
        self.encoder = models.resnet18()  
        self.projector = nn.Sequential([...])
```

5. Résultats

1. **Embeddings Visualisés** :
 - UMAP/tSNE démontrant la capacité à séparer des classes après entraînement.
2. **Précision supervisée en évaluation linéaire** :
 - Basé sur les représentations apprises : ~85 % dans notre cas.
3. **Graphiques** :
 - Courbes de pertes comparant entraînements contrastif et supervisé.

Exemples de résultats : - **Images Embeddings** : UMAP et t-SNE. - **Prédictions** : Pré-entraîné vs à partir de zéro (visuels inclus).

6. Challenges

- **Batch Size Limité** : Difficultés ajustées pour NT-Xent sur GPU mémoire réduite.
 - **Augmentations Non-Optimales** : Cache au chaud pour stratégies optimales.
-

7. Conclusion et Perspectives

Résultats

- Représentations robustes obtenues en utilisant un pipeline SimCLR autodéployé sur CIFAR-10.

Perspectives

- Améliorations possibles : backbone plus complexe (ResNet-50), apprentissage multi-échelle et dataset plus grand.
-